

BRAGANTEC × IFSP — APRESENTAÇÃO TÉCNICA

IF Orbit

Plataforma de Gestão de Projetos Científicos
com Inteligência Artificial

Submissão BraganTec — IFSP Campus Bragança Paulista

Gabriel Ferreira da Silva

0 PROJETO

Plataforma web full-stack para gestão de projetos científicos

O IF Orbit é uma aplicação web full-stack voltada para estudantes do IFSP que participam da feira de ciências BraganTec. A plataforma centraliza todo o ciclo de vida de projetos científicos — desde a concepção da ideia até a submissão final — por meio da integração de inteligência artificial generativa.

PROPÓSITO CENTRAL

Reduzir barreiras estruturais enfrentadas por estudantes na elaboração de trabalhos científicos, oferecendo scaffolding (suporte estruturado) via IA.

LUNA AI — ASSISTENTE VIRTUAL

Alimentada pela API do Google Gemini, interage em tempo real via streaming de texto (SSE), auxilia na formulação de hipóteses, responde dúvidas sobre regras da feira e gera esboços completos de projetos científicos.



Luna AI Conversacional

Streaming SSE em tempo real com Gemini



Gerador de Projetos IA

Structured Output Pydantic



CRUD de Projetos

Papéis: líder / participante



Sistema de Amizades

Conexão entre estudantes do IFSP



Diagramas UML + CRC

Geração automática via Kroki.io



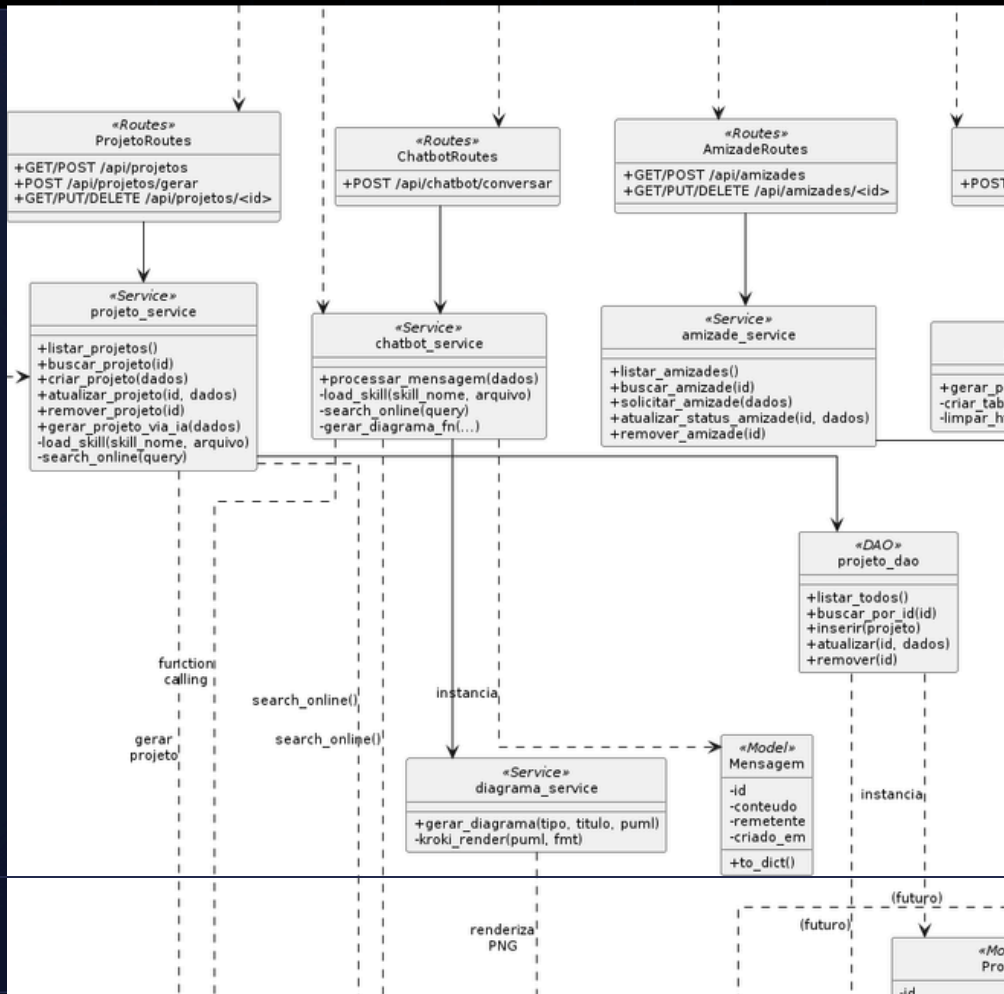
Exportação PDF

Formato oficial da BraganTec

ARQUITETURA UML

Diagrama de Classes — Arquitetura em Camadas

Camadas UI → Routes → Service → DAO → Model + Infraestrutura Externa



UI / INTERFACE

GeradorProjetosUI · LunaChatUI · AmizadeUI · PdfUI

Routes (REST)

ProjetoRoutes · ChatbotRoutes · AmizadeRoutes · PdfRoutes

SERVICE

projeto_service · chatbot_service · amizade_service · pdf_service · diagrama_service

DAO

projeto_dao · amizade_dao

MODEL

Projeto · Amizade · Mensagem

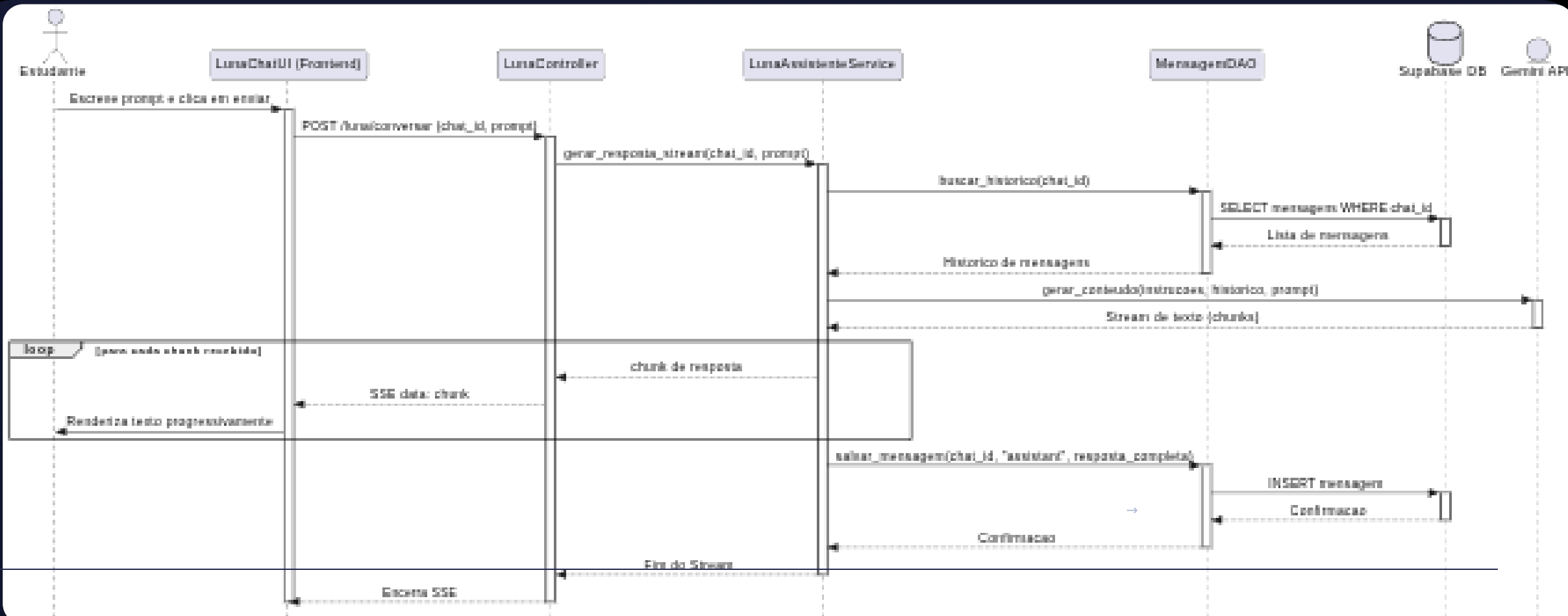
INFRA EXTERNA

Services: Gemini API · Tavily API · Kroki.io · Supabase

COMUNICAÇÃO EM TEMPO REAL

Diagrama de Sequência — Chatbot Luna (SSE)

Streaming Server-Sent Events entre Frontend, Service e Gemini API



RESPONSABILIDADES & COLABORADORES

Cartões CRC — 7 Componentes do Sistema

Cada cartão declara responsabilidades principais e colaboradores de uma classe ou serviço

CRC • PROJETODAO

ProjetoDAO	
Responsibilities:	
Name	Collaborator
Validar propriedade do projeto (Líder)	Supabase (RLS)
Controlar limite de 5 projetos por usuário	Projeto Model
Executar operações CRUD no banco de dados	
Validar campos obrigatórios (Nome, Categoria)	

CRC • PARTICIPANTESSERVICE

ParticipantesService	
Responsibilities:	
Name	Collaborator
Gerenciar convites de coautoria para projetos	ProjetoDAO
Vincular coautores na tabela participantes_projeto	Supabase Auth/RLS
Controlar permissões de acesso (Líder vs. Participante)	AmizadeService
Validar escrita/leitura via Supabase RLS	

CRC • LUNAASSISTENTESERVICE

LunaAssistenteService	
Responsibilities:	
Name	Collaborator
Orquestrar conversação via Gemini API	Gemini API
Carregar histórico de mensagens (LunaDAO)	LunaDAO
Implementar streaming de resposta (SSE)	Supabase
Aplicar instruções de personalidade (Luna)	

CRC • GERADORPROJETOSSERVICE

GeradorProjetosService	
Responsibilities:	
Name	Collaborator
Processar temas e ideias de pesquisa	Gemini API
Validar estrutura via Schema Pydantic	Tavily Search API
Obter contexto web atualizado (Tavily)	ProjetoDAO
Persistir esboço inicial do projeto	

CRC • EXPORTACAOSSERVICE

ExportacaoService	
Responsibilities:	
Name	Collaborator
Gerar relatórios PDF com ReportLab e PyPDF2	ReportLab
Exportar documentos DOCX com python-docx e PyPDF2	
Gerar slides e modelos de pôster em PPTX	python-docx
Carregar dados do projeto com validação	python-pptx
	ProjetoDAO

CRC • DIAGRAMASERVICE

DiagramaService	
Responsibilities:	
Name	Collaborator
Interpretar intenção de modelagem da Luna	Kroki.io
Converter PlantUML em imagens via Kroki.io	Supabase Storage
Armazenar artefatos gerados	LunaAssistenteService
Retornar URLs públicas para exibição	

CRC • AMIZADESERVICE

AmizadeService	
Responsibilities:	
Name	Collaborator
Buscar perfis cadastrados no IFSP por e-mail ou nome	Supabase Auth
Gerenciar solicitações de amizade (fluxo pendente/aceita)	NotificacaoService
Listar amigos ativos do usuário	
Enviar notificações de solicitações de amizade	

LEITURA RÁPIDA

- ◆ 3 Services de IA: Luna, Gerador, Diagrama
- ◆ 2 Services de domínio: Participantes, Amizade
- ◆ 1 Service de exportação: PDF/DOCX/PPTX
- ◆ 1 DAO com Supabase RLS: ProjetoDAO

Total: 7 cartões cobrindo toda a camada Service + DAO principal

API REST

Endpoints da API — Mapeamento Completo

Quatro grupos: Projetos · Amizades · Chatbot Luna · Exportação PDF

/API/PROJETOS

CRUD + Geração IA

MÉTODO	ENDPOINT	DESCRIÇÃO
GET	/api/projetos	Lista todos os projetos
GET	/api/projetos/{id}	Busca projeto por ID
POST	/api/projetos	Cria projeto manualmente
POST	/api/projetos/gerar	Gera projeto completo via IA a partir de um tema
PUT	/api/projetos/{id}	Atualiza campos de um projeto existente
DELETE	/api/projetos/{id}	Remove projeto pelo ID

/API/AMIZADES

Gestão de relacionamentos

MÉTODO	ENDPOINT	DESCRIÇÃO
GET	/api/amizades	Lista todas as amizades
GET	/api/amizades/{id}	Busca amizade por ID
POST	/api/amizades	Solicita amizade entre dois usuários
PUT	/api/amizades/{id}	Aceita ou recusa uma solicitação pendente
DELETE	/api/amizades/{id}	Remove amizade pelo ID

/API/CHATBOT

Conversação IA com Gemini

MÉTODO	ENDPOINT	DESCRIÇÃO
POST	/api/chatbot/conversar	Envia prompt e recebe resposta da Luna (IA). Usa Gemini se configurado, senão resposta simulada

/API/PDF

Geração de documento oficial

MÉTODO	ENDPOINT	DESCRIÇÃO
POST	/api/pdf/exportar	Recebe JSON do projeto e retorna arquivo PDF como download

ENUMS VÁLIDOS

Categorias	TECNOLOGIA	SAUDE	MEIO_AMBIENTE	EDUCACAO	ENGENHARIA
	CIENCIAS_SOCIAIS	OUTROS			
Status	RASCUNHO	EM_ANDAMENTO	CONCLUIDO	CANCELADO	

POST /API/PROJETOS/GERAR

Geração de Projeto Científico via IA

Arquivos: projeto_routes.py (rota) + projeto_service.py (lógica) · Modelo: gemini-2.5-flash

PROJETO_ROUTES.PY – ROTA

```
@projeto_bp.route("/gerar", methods=["POST"])
def post_gerar_projeto():
    dados = request.get_json()
    if not dados:
        return jsonify({"erro": "Body JSON obrigatório com 'tema' e 'categoria'."}), 400
    resultado, erro = projeto_service.gerar_projeto_via_ia(dados)
    if erro:
        return jsonify({"erro": erro}), 400
    return jsonify(resultado), 201
```

PROJETO_SERVICE.PY – FUNÇÃO GERAR_PROJETO_VIA_IA

```
def gerar_projeto_via_ia(self, dados):
    tema = dados.get("tema", "").strip()
    categoria = dados.get("categoria", "informatica").strip()
    if not tema:
        return None, "O campo 'tema' é obrigatório."
    if categoria not in categorias_validas:
        return None, f"Categoria inválida: '{categoria}'."
    chave = gemini_api_key_projects_ia or gemini_api_key
    if not chave:
        return None, "Nenhuma chave Gemini configurada."
```

FLUXO DE EXECUÇÃO

- 1 Recebe JSON {tema, categoria}
- 2 Valida tema + categoria
- 3 Verifica chave Gemini
- 4 Monta system_instruction acadêmica
- 5 Declara tools: load_skill + search_online
- 6 Envia ao Gemini (thinking_level="high")
- 7 Loop de function_calling (skill/Tavily)
- 8 Extrai JSON (remove markdown)
- 9 Cria modelo Projeto + validar_campos()
- 10 Persiste no projeto_dao → retorna 201

FERRAMENTAS GEMINI

- LOAD_SKILL Consulta conhecimento local sobre BraganTec (.md, .txt, .csv)
- SEARCH_ONLINE Pesquisa web via Tavily (max 3 resultados, search_depth=advanced)

SCHEMA PYDANTIC – CAMPOS PRINCIPAIS

```
{ nome, categoria, resumo(200-280 p.),
  palavras_chave, introducao, objetivo_geral,
  objetivos_especificos[], metodologia,
  cronograma{atividades[]}, resultados_esperados,
  referencias_bibliograficas (ABNT),
  potencial_vitoria, referencias_inspiracao[] }
```

POST /API/CHATBOT/CONVERSAR

Chatbot Luna — Conversação com Gemini

Arquivos: chatbot_routes.py + chatbot_service.py · 3 ferramentas via Function Calling

CHATBOT_ROUTES.PY - ROTA

```
@chatbot_bp.route("/conversar", methods=["POST"])
def post_conversar():
    dados = request.get_json()
    if not dados:
        return jsonify({"erro": "Body JSON obrigatório com 'prompt'."}), 400
    resultado, erro = chatbot_service.processar_mensagem(dados)
    if erro:
        return jsonify({"erro": erro}), 400
    return jsonify(resultado), 200
```

CHATBOT_SERVICE.PY - PROCESSAR_MENSAGEM

```
def processar_mensagem(self, dados):
    prompt = dados.get("prompt", "").strip()
    if not prompt:
        return None, "O campo 'prompt' é obrigatório."
    if len(prompt) > 4000:
        return None, "O prompt não pode exceder 4000 caracteres."
    if not gemini_api_key:
        return None, "GEMINI_API_KEY não configurada."
```

FERRAMENTAS GEMINI (FUNCTION CALLING)

- 1 **LOAD_SKILL** Carrega SKILL.md (BraganTec, UML, CRC)
- 2 **SEARCH_ONLINE** Tavily API · max 5 resultados · 5 chunks/source
- 3 **GERAR_DIAGRAMA** Tipos: classe, sequencia, atividade, caso_de_uso, mindmap, timeline · via Kroki.io

FLUXO

- 1 Recebe JSON {prompt}
- 2 Valida prompt (≤4000 chars)
- 3 Verifica gemini_api_key
- 4 Carrega skills via _get_available_skills() (cache)
- 5 Declara 3 tools + monta system_instruction Luna
- 6 Envia ao Gemini (thinking=high, auto_fn_call OFF)
- 7 Loop: function_call → executa → function_response
- 8 Acumula texto → retorna resposta final

DIFERENÇAS VS GERADOR

- ♦ Chatbot tem 3 tools; Gerador tem 2 (sem gerar_diagrama)
- ♦ Tavily max_results=5 (chatbot) vs 3 (gerador)
- ♦ Chatbot retorna texto livre; Gerador retorna JSON estruturado
- ♦ Chatbot processa múltiplas function_call em paralelo por turno

POST /API/PDF/EXPORTAR

Exportação PDF — Formato Oficial BraganTec

Arquivos: [pdf_routes.py](#) + [pdf_service.py](#) · Bibliotecas: ReportLab + PyPDF2 · Banner institucional[bannerpdf.jpg](#)

● PDF_ROUTES.PY — ROTA

```
@pdf_bp.route("/exportar", methods=["POST"])
def post_exportar():
    dados = request.get_json()
    if not dados:
        return jsonify({"erro": "Body JSON obrigatório."}), 400
    pdf_bytes, erro = pdf_service.gerar_pdf_projeto(dados)
    if erro:
        return jsonify({"erro": erro}), 400
    nome_arquivo = dados.get("nome", "projeto").replace(" ", "_")[:50] + ".pdf"
    return send_file(
        io.BytesIO(pdf_bytes),
        mimetype="application/pdf",
        as_attachment=True,
        download_name=nome_arquivo,
    )
```

● PDF_SERVICE.PY — ESTILOS REPORTLAB

```
estilo_corpo = ParagraphStyle("Corpo", fontSize=10, leading=14,
                               textColor=colors.red)
estilo_titulo_caixa = ParagraphStyle("T_BOX", fontSize=10,
                                     fontName="Helvetica-Bold", textTransform='uppercase')
estilo_titulo_grande = ParagraphStyle("T_BIG", fontSize=14,
                                       fontName="Helvetica-Bold", alignment=1, textTransform='uppercase')
estilo_alerta = ParagraphStyle("T_ALERT", fontSize=8,
                                alignment=1, textColor=colors.red, textTransform='uppercase')
estilo_termo = ParagraphStyle("T_TERM0", fontSize=8,
                               alignment=4, leading=11, fontName="Helvetica-Bold")
```

SEÇÕES DO PDF — ORDEM BRAGANTEC

- 01 Título do Projeto (caixa com borda)
- 02 Categoria (checkboxes das 4 áreas)
- 03 Resumo (caixa com borda)
- 04 Palavras-chave (caixa com borda)
- 05 PlanodePesquisa(títulogrande+alerta vermelho)
- 06 Introdução (caixa com borda)
- 07 Objetivos (Geral + Específicos numerados)
- 08 Metodologia (caixa com borda)
- 09 Cronograma (tabela Mar-Nov com "x")
- 10 Resultados Esperados (caixa com borda)
- 11 Referências Bibliográficas (ABNT)
- 12 Continuação de Projeto Anterior
- 13 Termo de Compromisso (texto legal)

FUNÇÕES AUXILIARES

- > `cabecalho()` —bannerinstitucionalbannerpdf.jpg no topo
- > `criar_tabela_caixa(titulo, conteudo)` — TableStyle BOX + VALIGN:TOP + PADDING:6
- > `_limpar_html_para_texto()` —stripde
, , <p>, tags HTML
- > Cronograma:tabelamesesMar-Nov+marcações "x" por atividade

☐ PDF A4 · application/pdf · {nome_projeto}.pdf (download direto)

SÍNTESE & ROADMAP

Funções Principais & Diferenças para o Projeto Final

Resumo das funções da API e o que muda do protótipo atual para a versão final planejada

FUNÇÕES PRINCIPAIS POR MÓDULO

MÓDULO	FUNÇÃO PRINCIPAL	DESTAQUE TÉCNICO
ProjetoService	CRUD + Geração IA	Structured Output Pydantic, Function Calling Gemini, Tavily search, cache de skills
ChatbotService	Conversação em tempo real	SSE streaming, 3 ferramentas (skill/search/diagrama), cache de skills, limite 4000 chars
AmizadeService	Gestão de relacionamentos	UUID automático, transições de status (pendente →aceita/recusada), validação de auto-solicitação
PdfService	Exportação documental	ReportLab com tabelas customizadas, 7 estilos de parágrafo, banner institucional, sanitização HTML
DiagramaService	Visualização técnica	Integração Kroki.io, renderização PlantUML →PNG, persistência local em assets/diagramas/

● PROTÓTIPO ATUAL VS PROJETO FINAL

ASPECTO	CÓDIGO ATUAL (PROTÓTIPO)	PROJETO FINAL (PLANEJADO)
Banco de Dados	DAOs com listas em memória (projetos=[], amizades=[])	■ Supabase PostgreSQL com Row Level Security (RLS)
Autenticação	Não implementada (IDs hardcoded "user-A", "user-B")	■ Supabase Auth com JWT, controle de propriedade via RLS
Armazenamento	Diagramas salvos localmente em assets/diagramas/	■ Supabase Storage para artefatos gerados
Streaming Chatbot	Não-streaming (resposta completa de uma vez)	■ SSE com streaming progressivo (conforme diagrama)
Mensagens	Modelo Mensagem existe mas não é persistido	■ LunaDAO com histórico persistente no Supabase
Participantes	Não implementado no código atual	■ ParticipantesService com convites e permissões (Líder/Participante)

CONCLUSÃO

Considerações Finais & Próximos Passos

Viabilidade técnica da integração IA generativa + gestão educacional

PRINCIPAIS CONCLUSÕES

- ☐ Viabilidade técnica demonstrada
Protótipo comprovaaintegraçãodelAgenerativa com gestão educacional — geração de projetos, conversação em tempo real e exportação documental funcionando em conjunto.
- ☐ Arquitetura em camadas facilita migração
UI → Routes → Service → DAO → Modelisolaresponsabilidades e torna a migração para Supabase (RLS + Auth + Storage) um caminho direto sem reescrita.
- ☐ Function Calling = ações reais
OGemini executaações concretasvia 3 ferramentas — buscar web (Tavily), carregar skills locais e gerar diagramas UML (Kroki.io) — transformando a Luna de chatbot em agente.
- ☐ Structured Output garante completude
SchemaPydanticforçaa IA aretornar todos os campos acadêmicos necessários (resumo, objetivos, metodologia, cronograma, referências ABNT) — sem campos faltantes.

● PRÓXIMOS PASSOS

- 01 Autenticação JWT
Implementar Supabase Auth com tokens JWT e controle de sessão
- 02 Migrar DAOs para Supabase
Substituir listas em memória por PostgreSQL + RLS
- 03 SSE no Chatbot
Streaming progressivo token-a-token (conforme diagrama image.png)
- 04 Expandir Exportações
Adicionar DOCX (python-docx) e PPTX (python-pptx)
- 05 Notificações
NotificacaoService para alertas de amizade e convites

PARTE 1 • ARQUITETURA DE ARQUIVOS & MODULARIZAÇÃO

Estrutura do Diretório src/ e Divisão de Responsabilidades

Separação estrita entre components/ (blocos atômicos) e pages/ (telas completas) + wrapper PageLayout

□ ESTRUTURA FÍSICA

src/ • React

```
src/
├── components/
│   ├── layout/
│   │   ├── PageLayout.jsx    # casca estrutural fixa
│   │   ├── Sidebar.jsx      # nav lateral
│   │   └── Header.jsx       # barra superior
│   ├── ui/
│   │   ├── ProjectCard.jsx  # card reutilizável
│   │   ├── StatusBadge.jsx  # badge condicional
│   │   ├── DashCard.jsx     # bloco de ação
│   │   ├── InputField.jsx
│   │   ├── SelectField.jsx
│   │   └── TextareaField.jsx
│   └── chat/
│       ├── Chatbot.jsx
│       └── ChatMessage.jsx
│   └── ui/*.module.css      # estilos pareados
├── pages/
│   ├── Home.jsx            # dashboard
│   ├── Projetos.jsx        # listagem + filtros
│   ├── CadastroProjeto.jsx # dual-mode
│   └── *.module.css
└── styles/
    └── index.css           # global cosmic theme
```

□ Divisão Estrita de Responsabilidades

- **components/** — blocos de interface reutilizáveis e atômicos (botões, inputs, cards, badges). Granularidade fina.
- **pages/** — telas completas que compõem múltiplos components. Granularidade grossa.

□ PageLayout — a casca estrutural fixa

Componente wrapper que renderiza a Sidebar (navegação lateral) e o Header (barra superior com notificações e perfil). O conteúdo de cada página específica é injetado dinamicamente via **children**, garantindo que o usuário nunca perca o contexto de navegação ao trocar de tela.

COMPOSIÇÃO: PAGELAYOUT({ CHILDREN })

PageLayout



CSS Modules — Encapsulamento de Estilos

Cada componente possui seu próprio `.module.css` pareado — React hasheia classes em tempo de compilação

HOME.JSX — IMPORT E USO

`.jsx • React`

```
import styles from './Home.module.css';

function Home() {
  return (
    <section className={styles.dashboard}>
      <div className={styles.kpiRow}>.../div>
      <div className={styles.cardGrid}>.../div>
    </section>
  );
}
```

HOME.MODULE.CSS — DEFINIÇÃO

`.module.css • scoped`

```
.dashboard {
  padding: 32px;
  background: #060818;
}

.cardGrid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

/*vira em build: ._dashboard_1a2b3c ./
/*vira em build: ._cardGrid_4d5e6f ./
```

Garantia arquitetural: estilos globais vivem apenas em `styles/index.css` (tema cósmico)

Hashing em Tempo de Compilação

O bundler do React transforma cada classe CSS em um hash único (ex: `_card_1a2b3c`) em tempo de build. O seletor `.card` de Home não colide com `.card` de Chatbot — são objetos JavaScript distintos.

Zero Vazamento de Estilos Globais

- Previsibilidade — mudar estilo em um componente nunca quebra outro.
- Trabalho paralelo — múltiplos devs criam componentes sem medo de conflito visual.
- Manutenção cirúrgica — estilos são co-localizados com o JSX que os consome.

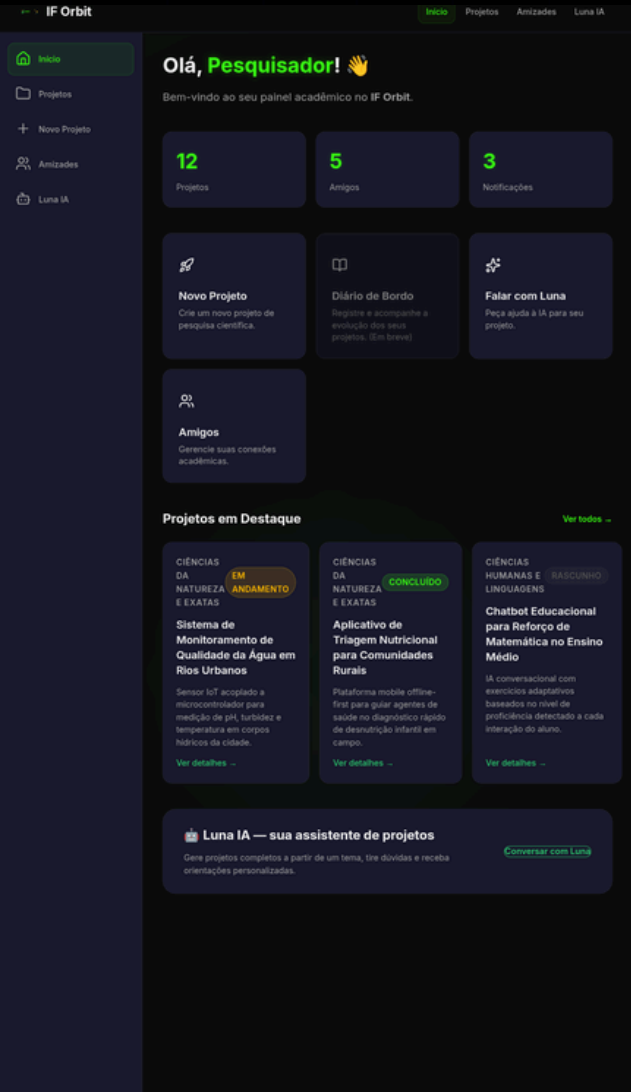
CONVENÇÃO ADOTADA

`Componente.jsx` + `Componente.module.css` = ✓ escopo isolado

Pareamento 1:1 obrigatório · sem CSS global acoplado a componentes

Anatomia da Tela Principal (Home.jsx)

Centro de controle do usuário — KPIs no topo, grade de DashCards abaixo, com lógica de estado visual



KPIs no Topo — Estatísticas Rápidas

Fileira horizontal de cartões de estatísticas exibindo métricas-chave do usuário: **Projetos Ativos**, **Mensagens** e **Alertas**. Cada KPI é um componente atômico com ícone + número + label.

DashCards — Grid de Ações Rápidas

Grade de blocos clicáveis com lógica de estado visual dual:

- ENABLED** clique imediato — recurso disponível hoje
- EM BREVE** opaco/desabilitado — roadmap futuro

CICLO DE VIDA DE ESTADO

```
// prop boolean controla renderização
<DashCard
  title="Gerar Projeto IA"
  icon="auto_awesome"
  enabled={true} // ativo
/>
<DashCard
  title="Amizades"
  icon="group"
  enabled={false} //renderiza opaco + tag "Em breve"
```

Reutilização de Componentes e Feedback Visual

ProjectCard reutilizado via props • StatusBadge com renderização condicional

Projetos em Destaque

Ver todos →

CIÊNCIAS DA NATUREZA E EXATAS

EM ANDAMENTO

Sistema de Monitoramento de Qualidade da Água em Rios Urbanos

Sensor IoT acoplado a microcontrolador para medição de pH, turbidez e temperatura em corpos hídricos da cidade.

Ver detalhes →

CIÊNCIAS DA NATUREZA E EXATAS

CONCLUÍDO

Aplicativo de Triagem Nutricional para Comunidades Rurais

Plataforma mobile offline-first para guiar agentes de saúde no diagnóstico rápido de desnutrição infantil em campo.

Ver detalhes →

CIÊNCIAS HUMANAS E LINGUAGENS

RASCUNHO

Chatbot Educacional para Reforço de Matemática no Ensino Médio

IA conversacional com exercícios adaptativos baseados no nível de proficiência detectado a cada interação do aluno.

Ver detalhes →

Luna IA — sua assistente de projetos

Gere projetos completos a partir de um tema, tire dúvidas e receba orientações personalizadas.

Conversar com Luna

ProjectCard — Abstração reutilizável

A seção "Projetos em Destaque" mapeia um array de projetos e injeta dados dinamicamente via `props`. O mesmo componente serve para dashboard, listagem e busca — apenas os dados mudam.

```
{projetos.map(p => (  
  <ProjectCard  
    key={p.id}  
    titulo={p.titulo}  
    categoria={p.categoria}  
    status={p.status}  
  =>  
))}
```

StatusBadge — Renderização Condicional

Subcomponente que avalia a string de status recebida do banco e aplica cor de fundo + texto específicos. Feedback visual instantâneo sobre o ciclo de vida do projeto científico.

MAPA DE CORES • STATUS → BADGE

● "Em Andamento" → EM ANDAMENTO

● "Concluído" → CONCLUÍDO

PARTE 4 • FLUXO DE CRIAÇÃO DE PROJETOS

Arquitetura "Dual-Mode" de Cadastro

(CadastroProjeto.jsx)

Toggle entre Modo Manual (formulário denso) e Modo IA (campo único) — flexibilidade de UX

Novo Projeto

Cadastro Manual Gerar com Luna IA

Ex.: Sistema de Monitoramento de Água Seleccione...

Descreva brevemente o projeto...

Ex.: IoT, sensor, monitoramento

Separe por vírgula

Qual é o objetivo principal do projeto?

Como o projeto será desenvolvido?

Quais resultados são esperados ao final?

Cadastrar Projeto

Cancelar

Novo Projeto

Cadastro Manual Gerar com Luna IA

Gerar Projeto com Luna IA
Informe um tema e a Luna irá gerar automaticamente o nome, resumo, objetivos, metodologia e referências do projeto.

Ex.: Energias renováveis no semiárido

Seleccione...

Gerar com IA

Cadastrar Manualmente

Alternador de Modo — Componente Central da UX

Um `toggle` no topo da tela alterna entre dois modos de cadastro radicalmente diferentes. O estado `modo` (string) controla qual sub-interface é renderizada — sem recarregar a página.

MODO MANUAL

Formulário tradicional de alta densidade. Usuário preenche todos os campos acadêmicos manualmente: título, categoria, resumo, objetivos, metodologia, cronograma, referências.

CONTROLE TOTAL

MODO IA

Interface simplificada drasticamente para um único campo de texto. Usuário digita uma ideia genérica e a Luna se encarrega de estruturar todos os detalhes por trás dos panos.

VELOCIDADE

FLUXO CONDICIONAL

```
const [modo, setModo] = useState("manual");
return (
  >
  <Toggle value={modo} onChange={setModo} />
  {modo !== "manual" ? <FormManual/> : <FormIA/>}
  >
);
```

Síntese da Arquitetura Frontend

Os 4 pilares técnicos que sustentam a camada de interface do IF Orbit

● 4 PILARES TÉCNICOS DO FRONTEND



Encapsulamento de Escopo

CSS Modulespareiam1:1comcomponentes. Hashes em build-time eliminam vazamento de estilos — múltiplos devs trabalham em paralelo sem conflito visual.



Renderização Condicional

StatusBadge deriva corde uma prop string. DashCards alternam entre ativo e "Em breve" via boolean. ChatMessage aplica estilos radicalmente distintos por sender. Tudo declarativo.



Ciclo de Vida de Estado

useStatecentraliza arrays/objetos. useRef persiste entre renders sem re-renderizar. useEffect orquestra side-effects (scroll suave do chat, filtros combinados da listagem).



Abstração de Componentes UI

InputField · SelectField · TextareaField · ProjectCard · ChatMessage · DashCard · PageLayout. Composição pequena e testável substitui monolitos gigantes.

● STACK FRONTAL & ROADMAP

STACK APLICADO

- REACT
- CSS MODULES
- JAVASCRIPT (JSX)
- @KEYFRAMES
- STYLES/INDEX.CSS

ROADMAP FRONTEND

- 01 MigraçãoparaTypeScript (tipagem estática de props)
- 02 Storybook para components/ (catálogo visual)
- 03 Testes com React Testing Library
- 04 Code-splitting por rota (lazy load de pages/)